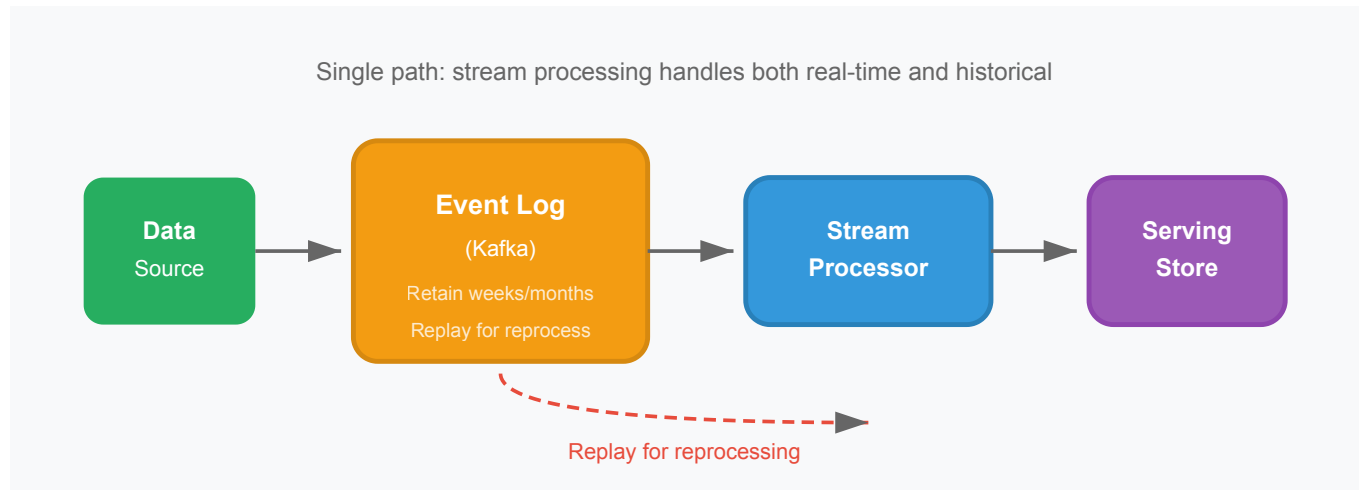


Kappa Architecture

 systemdesignschool.io/fundamentals/kappa-architecture



The previous article showed Lambda architecture's approach: run batch and stream in parallel, merge results. It works, but you maintain two implementations of the same logic. Jay Kreps, co-creator of Apache Kafka, proposed a simplification: **if you have a replayable event log, batch processing is just stream processing replayed from the beginning.**

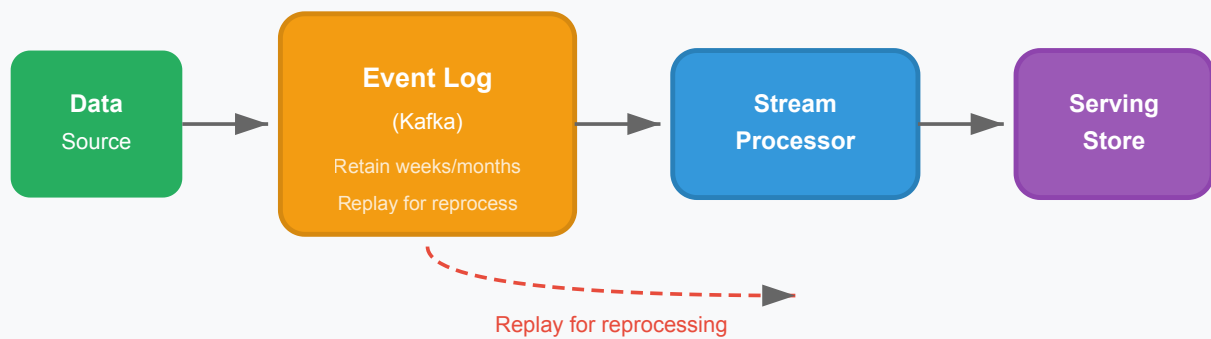
Instead of separate batch and stream pipelines, run everything through streaming. When you need to reprocess—for bug fixes, schema changes, or new features—replay the event log through the same streaming code.

The Core Idea

The event log (typically Kafka or a similar system) serves as the system of record. It retains all events for a configurable period—days, weeks, or longer depending on storage and retention policy.

Stream processing applications consume from the log, apply transformations, and write results to serving stores. There's no separate batch layer. The "batch" capability comes from replaying the log.

Single path: stream processing handles both real-time and historical



Reprocessing Through Replay

Suppose you discover a bug in your revenue calculation logic. In Lambda architecture, you'd fix the bug in both the batch codebase and the stream codebase, then rerun the batch job.

In Kappa architecture:

1. Fix the bug in your single stream processing application
2. Deploy a new instance that starts reading from the beginning of the log
3. The new instance replays all historical events, applying the corrected logic
4. Once it catches up to the present, swap it in as the active processor
5. Shut down the old instance

This matters because reprocessing follows the exact same code path as live processing. One implementation, one test suite, one deployment pipeline. You don't have subtle differences between batch and stream versions causing result discrepancies.

Why Modern Infrastructure Makes This Feasible

Kappa architecture became practical when message brokers evolved beyond traditional queue semantics.

Durable, long-term storage. Kafka retains events on disk with configurable retention. A log with weeks or months of retention is effectively an append-only database. You don't need to extract data to a separate batch store.

Independent consumption. Multiple consumer groups read the same log independently. A reprocessing job can replay from offset 0 while the live job continues processing at the current offset. They don't interfere with each other.

Scalable replay. Parallel consumers across partitions mean replay can run as fast as your cluster allows—limited by disk I/O and compute, not by the broker's design.

A Concrete Example

Consider a real-time dashboard showing "orders per hour" by region.

Initial deployment: Stream processor reads from Kafka, computes hourly aggregates, writes to a serving database. Dashboard queries the database.

Bug discovered: Region codes were incorrectly mapped for Canadian orders. All "CA" orders were counted as California instead of Canada.

Kappa reprocessing:

1. Fix the region mapping in the stream processor code
2. Deploy new instance: `stream-processor-v2`, configured to start from offset 0
3. v2 replays the entire log with correct logic, writing to a new table `orders_by_region_v2`
4. Once v2 catches up (lag near zero), update dashboard to query `orders_by_region_v2`
5. Decommission v1 and drop the old table

No separate batch codebase. No batch cluster. The fix applies identically to historical and future data.

Trade-offs and Limitations

When Kappa works well:

- Events are the primary source of truth
- Reprocessing time is acceptable for your use case
- Business logic is identical for real-time and historical processing
- Your event log retention covers the history you need

When Kappa struggles:

- Historical data predates the event log (you can't replay what was never captured)
- Reprocessing takes too long (petabytes of events might take days to replay)
- Some computations genuinely require batch semantics (certain ML training workflows, complex joins across large historical datasets)
- External systems aren't designed for reprocessing (sinks that can't handle replayed data cleanly)

Lambda vs Kappa: Choosing

Factor	Lambda	Kappa
Codebases	Two (batch + stream)	One
Infrastructure	Two clusters	One

Factor	Lambda	Kappa
Reprocessing mechanism	Run batch job	Replay event log
Operational complexity	Higher	Lower
Historical data	Can process data outside the log	Limited to log retention
Best for	Compliance-critical systems, existing batch investments	Greenfield projects, event-sourced architectures

For new projects with modern infrastructure and event-sourcing principles, Kappa often wins on simplicity. For enterprises with significant batch investments, strict compliance requirements tied to existing batch systems, or historical data that was never event-captured, Lambda provides a path to add real-time without disrupting proven systems.

What's Next

Both Lambda and Kappa treat batch and stream as somewhat separate concerns. **Unified batch-stream frameworks**—modern systems like Apache Flink and Spark Structured Streaming—offer a single API for both bounded and unbounded data, blurring the distinction further.